Interview Coding Exercise

March 20, 2026

You will implement a small project to evaluate basic coding skills. Do not worry, this task is not intended to be difficult or complicated, and you are free to use any tools you want, including LLMs. The goal is to produce a clean and well-structured project repository that can be installed as a Python package. The following sections describe the tasks you are expected to complete. In some cases, there is no single correct solution, and implementation choices may depend on your preferred coding style. We recommend reading through this document once to understand the overall goal before starting so you can structure your code accordingly.

1 Introduction

A 3D Scene Graph (3DSG) [AHG+19, HCC22] is a semantically rich and compact representation of a scene. It is commonly represented as a hierarchical structure with multiple layers of abstraction, such as a root node, rooms, objects, and object parts 1. Each node contains semantic information and relationships to other nodes in the graph. This representation is often used in robotics and computer vision to reason about environments in a structured way.

2 Implementation

2.1 Setup

Your first task is to create a new repository for this project. You will share this repository with us, including the entire commit history. Create a virtual environment using Python 3.10 and prepare a `pyproject.toml` file. Your project should be structured as a proper Python package that can be installed using `pip`. Push an initial commit containing a simple README file with a brief description of a 3D Scene Graph and an illustration of the concept. You are free to choose the name of the project and the package as long as it is descriptive.

For each of the following steps, create a new experimental branch when making changes. Once you are satisfied with the results, merge the branch into main. Ensure that all external libraries used in the project are listed in the

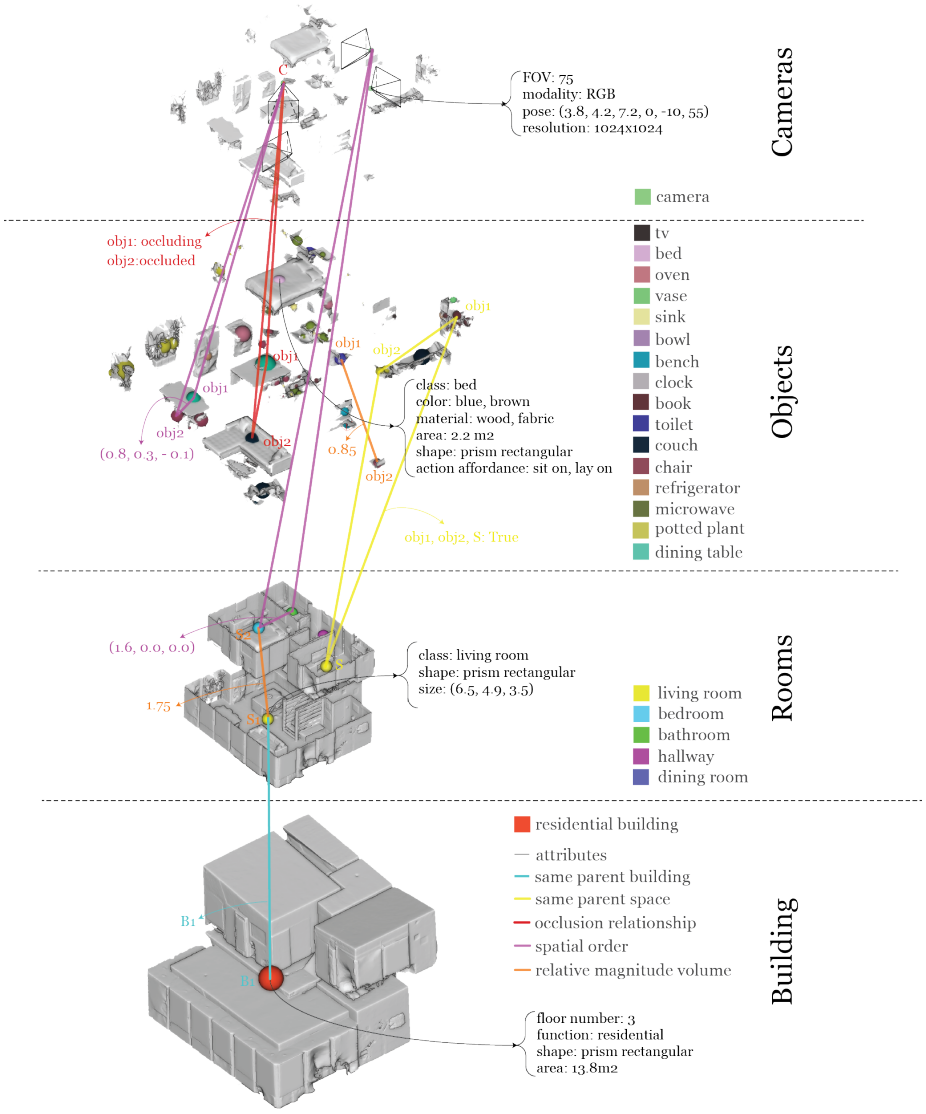

Figure 1: An example of a 3DSG from [AHG+19].

`pyproject.toml` file.

2.2 3DSG Class

Implement a class called `SceneGraph` that can be constructed from a nested dictionary describing a building. An example of such a dictionary will be provided below. For each node type (e.g., `Room`, `Object`, `Frontier`, etc.), implement a separate class with attributes and methods that you consider appropriate for representing the data and relationships within the scene graph.

2.3 Loading 3DSG from JSON Files

The `SceneGraph` class should implement a class method called `load_from_json` that loads an instance of the graph from a JSON file.

Create a file called `main.py` in the root directory of your project. In this file, import your `SceneGraph` class and provide an example demonstrating how to load a graph from a JSON file and print some basic properties or statistics of the graph.

2.4 Loading 3DSG from an Image

In this section, your task is to construct a scene graph from a single input image.

You will need to load a segmentation model (for example SAM) and choose a set of object categories to segment. The goal is to generate a scene graph containing a root node, a room node, objects, and optionally object parts extracted from the image.

This section is intentionally open-ended. There is no single correct approach, and we encourage you to be creative in how you design the pipeline. You may select any indoor household image and attempt to extract objects and relationships from it to populate the graph structure.

We provide some example code below showing how to run SAM3 for multiple classes as inspiration based on huggingface implemetation. You are free to modify or extend this approach. Depending on your GPU and environment, you may need to adapt the code. Execution time is not critical for this exercise.

Add a new example in `main.py` demonstrating how to generate a scene graph from an image.

```
import matplotlib

import torch
```

```python
import numpy as np
from PIL import Image

from transformers import Sam3Processor, Sam3Model


def overlay_masks(image, masks):
    image = image.convert("RGBA")
    masks = 255 * masks.cpu().numpy().astype(np.uint8)

    n_masks = masks.shape[0]
    cmap = matplotlib.colormaps.get_cmap("rainbow").resampled(n_masks)
    colors = [
        tuple(int(c * 255) for c in cmap(i)[:3])
        for i in range(n_masks)
    ]

    for mask, color in zip(masks, colors):
        mask = Image.fromarray(mask)
        overlay = Image.new("RGBA", image.size, color + (0,))
        alpha = mask.point(lambda v: int(v * 0.5))
        overlay.putalpha(alpha)
        image = Image.alpha_composite(image, overlay)
    return image


device = "cuda" if torch.cuda.is_available() else "cpu"

model = Sam3Model.from_pretrained("facebook/sam3").to(device)
processor = Sam3Processor.from_pretrained("facebook/sam3")
image = Image.open("table_runner.jpg")

# Try both lists to see results
text_prompts = ["dining room","kitchen", "bathroom", "bedroom"]
text_prompts = ["table", "table legs", "table top"]

images = [image] * len(text_prompts)

inputs = processor(images=images, text=text_prompts, return_tensors="pt").to(device)

with torch.no_grad():
    outputs = model(**inputs)

# Post-process results for both images
results = processor.post_process_instance_segmentation(
    outputs,
```

```
        threshold=0.5,
        mask_threshold=0.5,
        target_sizes=inputs.get("original_sizes").tolist()
    )

    print(f"Image 1: {len(results[0]['masks'])} objects found")
    print(f"Image 2: {len(results[1]['masks'])} objects found")

    overlay_masks(image,results[0]["masks"])
```

2.5 Bonus

This section is optional. Here, you are encouraged to explore an additional idea related to the project and extend the implementation.

Examples could include:

- Trying a different segmentation model
- Adding additional abstraction layers in the graph
- Including spatial relationships between rooms or objects
- Adding visualization tools for the scene graph

If you implement an extension, update the `main.py` file with an example demonstrating the new functionality and update the README file with a short description of what was added or an interpretation of results if applicable.

References

[AHG+19] Iro Armeni, Zhi-Yang He, JunYoung Gwak, Amir R Zamir, Martin Fischer, Jitendra Malik, and Silvio Savarese. 3d scene graph: A structure for unified semantics, 3d space, and camera. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5664–5673, 2019.

[HCC22] Nathan Hughes, Yun Chang, and Luca Carlone. Hydra: A real-time spatial perception system for 3d scene graph construction and optimization. *arXiv preprint arXiv:2201.13360*, 2022.